

Abstract Data Types: Foundations of Modern Software Design

Understanding the Separation Between
Interface and Implementation

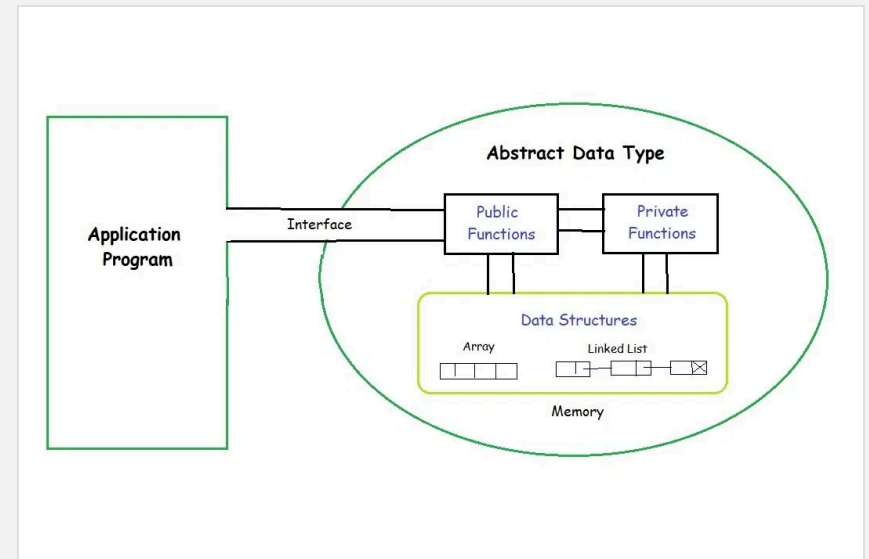
**COMPE496 Algorithms
and Programming**

Murat Balci
Spring 2026

Managing Complexity Through Abstraction Enables Robust Software Systems

Modern software systems contain millions of lines of code that would be impossible to understand without abstraction mechanisms. ADTs separate **what** operations can be performed from **how** those operations are implemented.

- **Abstraction:** Hides low-level details behind simpler, higher-level concepts.
- **Modularity:** Divides systems into independently designed and tested components.
- **Encapsulation:** Protects internal behavior from external interference.
- **Information Hiding:** Allows implementation changes without affecting clients.



The interface acts as a contract, shielding the application program from the complexity of the underlying data structures.

ADTs Define Types by Their Operations, Not Their Implementation

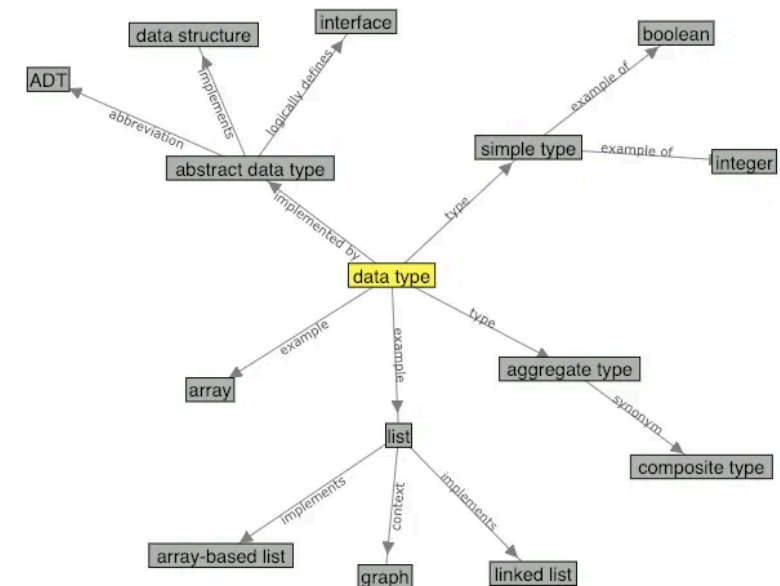
FORMAL DEFINITION

An ADT consists of a type name, a set of operations, and specifications for each operation that define behavior through inputs and outputs.

It is a mathematical model that specifies a data type purely through its behavior from the user's perspective. The implementation details—memory layout and algorithms—remain hidden.

Historical Significance

Pioneered by Barbara Liskov and John Guttag; foundational to modern object-oriented design.



The conceptual relationship between Abstract Data Types, logical definitions, and physical data structures.

ADTs Separate Logical Behavior from Physical Implementation

Logical View (Interface)

The "What". Defined by the set of operations and their behavior. The user interacts with this layer without knowing the internal structure.

```
// std::vector interface  
std::vector<int> v;  
v.push_back(10);  
int x = v.front();
```

Physical View (Implementation)

The "How". The specific data structures, memory layout, and algorithms used to realize the behavior. Hidden from the user.

```
// Inside std::vector  
int* _start;  
int* _finish;  
int* _end_of_storage;  
// Dynamic memory allocation
```

Mutability vs. Immutability in C++ ADTs

| Mutable Types

Objects whose state can be modified after creation. C++ objects are mutable by default unless marked `const`.

```
std::string s = "Hello";  
s[0] = 'h'; // Modifies in place  
s.append(" World");
```

CHARACTERISTICS

- Efficient (no copying needed)
- Harder to reason about (aliasing risks)
- Requires synchronization in threads

| Immutable Types

Objects whose state cannot change. In C++, we enforce this via `const` correctness or design.

```
const std::string s = "Hello";  
// s[0] = 'h'; // Compile Error!  
std::string s2 = s + " World";
```

CHARACTERISTICS

- Safe from aliasing bugs
- Thread-safe by default
- May require copying (performance cost)

Classifying ADT Operations

CREATORS

Create new objects of the type. Can be constructors or static factory methods.

```
std::vector<int> v; // Default ctor  
auto p = std::make_unique<T>();
```

PRODUCERS

Create new objects from existing ones. The original object is unchanged.

```
std::string s = "foo";  
std::string s2 = s + "bar"; // operator+
```

MUTATORS

Change the state of an object. Only available for mutable types.

```
std::vector<int> v;  
v.push_back(10); // Modifies v  
v.clear();
```

OBSERVERS

Inspect the state of an object without modifying it. In C++, these are const methods.

```
bool empty() const;  
size_t size() const;
```

Common ADTs in the C++ Standard Template Library (STL)

List / Sequence

`std::vector`, `std::list`

Ordered collection of elements. Supports indexed access (vector) or efficient insertion (list).

Set

`std::set`, `std::unordered_set`

Collection of unique elements. Supports membership testing. Ordered (Tree) or Unordered (Hash).

Map / Dictionary

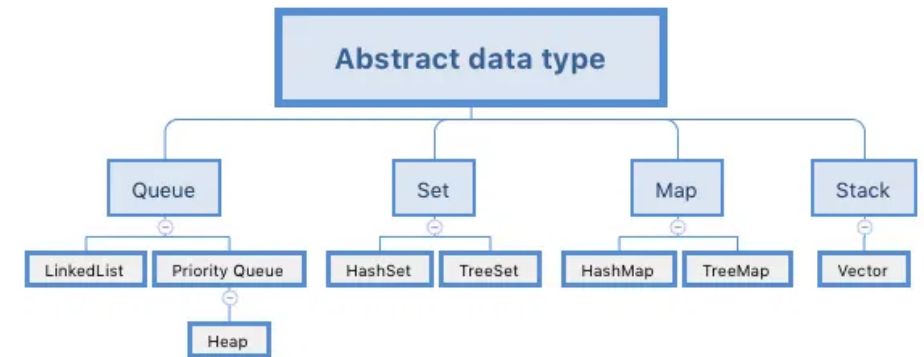
`std::map`, `std::unordered_map`

Collection of Key-Value pairs. Keys are unique. Essential for lookups.

Container Adapters

`std::stack`, `std::queue`

Restricted interfaces built on top of other containers to enforce specific access patterns.



The STL provides concrete implementations (Vector, Set) for abstract concepts. Note how "Adapters" wrap underlying containers.

Same ADT, Different Implementations

A single ADT (like a List) can have multiple implementations with vastly different performance characteristics.

`std::vector` (Dynamic Array)

Random Access	$O(1)$
---------------	--------

Insert at End	$O(1)$ Amortized
---------------	------------------

Insert at Front	$O(n)$
-----------------	--------

```
v[5] = 10; // Direct memory offset
```

Cache Locality: Excellent. Elements are contiguous in memory.

`std::list` (Doubly Linked)

Random Access	$O(n)$
---------------	--------

Insert at End	$O(1)$
---------------	--------

Insert at Front	$O(1)$
-----------------	--------

```
l.insert(it, 10); // Pointer manipulation
```

Cache Locality: Poor. Nodes are scattered in heap memory.

Designing Robust C++ ADTs

RAII (Resource Acquisition Is Initialization)

RESOURCE MANAGEMENT

The ADT must acquire resources (memory, file handles) in the constructor and release them in the destructor. This guarantees no leaks, even on exceptions.

```
~MyADT() { delete[] data; }
```

The Rule of Five

COPY CONTROL

If your ADT manages a resource, you must define (or delete) the Destructor, Copy Constructor, Copy Assignment, Move Constructor, and Move Assignment.

```
MyADT(const MyADT& other); operator=(MyADT&& other);
```

Interface Segregation

ENCAPSULATION

Keep the public header file clean. Use the Pimpl (Pointer to Implementation) idiom to hide private members and dependencies from the client.

```
class Impl; std::unique_ptr<Impl> pImpl;
```

Representation Independence

You should be able to change the internal data structures (the representation) without breaking any client code that uses the ADT.

Implementation V1 (Vector)

```
class IntSet {  
public:  
    void add(int x) { v.push_back(x); }  
private:  
    std::vector<int> v;  
};
```

Implementation V2 (Set)

```
class IntSet {  
public:  
    void add(int x) { s.insert(x); }  
private:  
    std::set<int> s;  
};
```

```
IntSet mySet;  
mySet.add(42); // Unchanged!
```

Note: In C++, changing private members changes the object layout, requiring client *recompilation*, but not *rewriting*.

Invariants: The Soul of the ADT

Representation Exposure in C++

Common Causes

Returning Non-Const References

Allows clients to modify internal state directly, bypassing invariants.

```
std::vector<int>& getList() { return v; }
```

Returning Pointers

Gives raw memory access. Can lead to dangling pointers if the ADT is destroyed.

```
int* getData() { return &data; }
```

Prevention Strategies

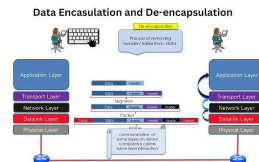
Const Correctness

Return const references to provide read-only views without copying.

```
const std::vector<int>& getList() const;
```

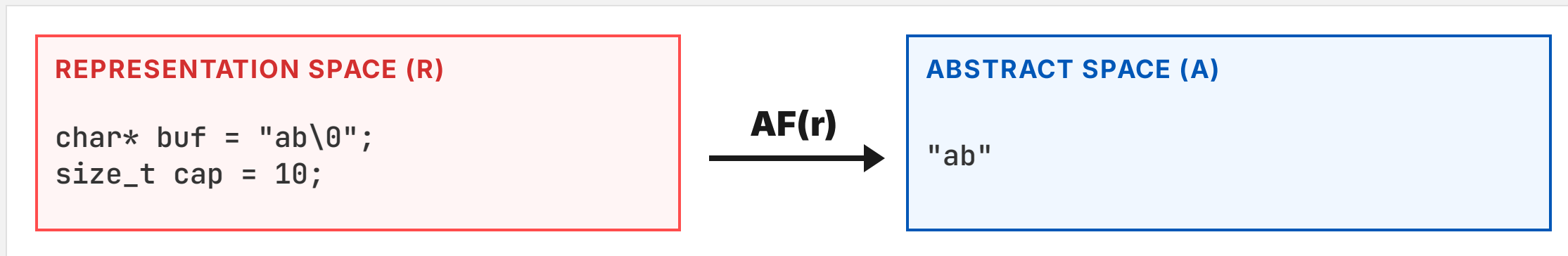
Return by Value

Return a copy of the data. Safer but potentially slower (mitigated by Move Semantics).



Visual Metaphor: Just as networking layers encapsulate data, ADTs must protect their internal memory layout.

Abstraction Functions Map Representations to Abstract Values



Many-to-One

Multiple concrete representations can map to the same abstract value.

```
buf="ab\0", cap=10  
buf="ab\0", cap=20  
Both map to "ab"
```

Partial Function

Not all concrete states are valid. The AF is only defined for states satisfying the Representation Invariant.

```
buf = nullptr  
(Undefined behavior)
```

Surjective

Every possible abstract value must be producible by at least one concrete representation.

```
For any std::string s,  
there exists a char*
```

Representation Invariants Define Legal Representations

Mathematical Definition

$RI: R \rightarrow \text{boolean}$

The Rep Invariant is a predicate that partitions the universe of all possible concrete values into two sets: those that represent valid abstract values and those that do not.

INVALID STATE (RI = FALSE)

Valid State (RI = true)

Example: Sorted List
 $\text{list}[0] \leq \text{list}[1] \leq \dots \leq \text{list}[n]$

Example: [1, 5, 2] (Unsorted)

Runtime Enforcement

In C++, we implement the invariant as a private method, often conditionally compiled for debugging.

```
void checkRep() {  
    #ifndef NDEBUG  
        // Check structural property  
        assert(count ≥ 0);  
        assert(count ≤ capacity);  
  
        // Check complex relationship  
        if (count > 0) {  
            assert(data ≠ nullptr);  
        }  
    #endif  
}
```

- ✓ Call at end of all **constructors**
- ✓ Call at end of all **mutators**
- ✓ Use `assert()` to fail fast in debug builds

ADT Invariants Replace Preconditions and Simplify Specifications

By shifting responsibility from the client (caller) to the ADT (implementer), we create types that are safer and easier to use correctly.

Formal Specifications: The Contract

```
/**
 * @brief Removes the element at index i.
 *
 * @param i The index to remove.
 * @pre 0 ≤ i < size()
 * @post size() decreases by 1.
 * @post Elements at j > i shift to j-1.
 * @throw std::out_of_range if i is invalid.
 */
void remove(size_t i);
```

PRECONDITIONS (@PRE)

Constraints on the client. If violated, the behavior is undefined (unless checked by exceptions).

POSTCONDITIONS (@POST)

Promises made by the implementer. Describes the new state and return value relative to the input.

FRAME CONDITIONS

What does *not* change? (Implicitly: other elements in the list remain modified only by the shift).

Testing ADTs: Strategies & Tools

Black Box Testing

Test only against the public specification. Do not assume anything about the implementation details.

Glass Box Testing

Test internal state and invariants. In C++, use `friend` class to grant test fixtures access to private members.

Example Test Case

GoogleTest (gtest)

```
TEST(IntSetTest, AddIncreasesSize) {  
    // Setup  
    IntSet s;  
  
    // Action  
    s.add(10);  
    s.add(20);  
  
    // Assertion (Black Box)  
    EXPECT_EQ(s.size(), 2);  
    EXPECT_TRUE(s.contains(10));  
  
    // Assertion (Glass Box via friend)  
    // EXPECT_LE(s.capacity, 100);  
}
```

Defining Interfaces with Abstract Base Classes

Pure Virtual Functions

In C++, an interface is a class containing only pure virtual functions ($= 0$). It defines *what* needs to be done, not *how*.

CRITICAL: Always declare a virtual destructor in the base class to ensure derived classes are properly cleaned up when deleted via a base pointer.

```
class List {
public:
    // Virtual Destructor (Essential!)
    virtual ~List() = default;

    // Pure Virtual Functions
    virtual void add(int x) = 0;
    virtual int get(size_t i) const = 0;
};

class ArrayList : public List {
public:
    void add(int x) override { ... }
    int get(size_t i) const override { ... }
};
```

Generic ADTs with C++ Templates

Compile-Time Polymorphism

Templates allow you to write code once and use it with any type. Unlike Java generics (type erasure), C++ generates specialized code for each type.

Zero Overhead: No runtime cost for type checking.

Type Safety: Errors caught at compile time.

Flexibility: Works with primitives (int) and objects.

```
template <typename T>
class List {
public:
    void add(const T& element);
    T get(size_t index) const;
private:
    std::vector<T> data;
};

// Usage
List<int> intList;
List<std::string> strList;
```

ADTs in the Real World: Performance Matters

Chromium Browser

CACHE LOCALITY

Google's Chromium project explicitly discourages `std::list`. They favor `std::vector` almost exclusively because contiguous memory access is significantly faster on modern CPUs due to cache prefetching.

KEY ADT

```
std::vector<T> (Dynamic Array)
```

LLVM Compiler

MEMORY OPTIMIZATION

LLVM defines custom ADTs like `SmallVector` (stack-allocated for small sizes) and `DenseMap` (quadratic probing) to reduce heap allocations and pointer chasing during compilation.

KEY ADT

```
llvm::SmallVector<T, N>
```

Common ADT Design Patterns

Iterator Pattern

Provides a standard way to traverse a collection without exposing its underlying representation. In C++, iterators generalize pointers.

```
std::vector<int> v = {1, 2, 3};  
for (auto it = v.begin(); it != v.end();  
    ++it) {  
    std::cout << *it << " ";  
}
```

Adapter Pattern

Converts the interface of a class into another interface clients expect. `std::stack` adapts a sequence container (like `deque`) to provide LIFO behavior.

```
// Stack adapting a vector  
std::stack<int, std::vector<int>> s;  
s.push(10); // Calls vector::push_back  
s.pop(); // Calls vector::pop_back
```

Performance: Theory vs. Reality

Theory: Big-O Complexity

Asymptotic analysis focuses on operation counts as N grows.

Linked List Insert: $O(1)$

Vector Insert (middle): $O(N)$

Reality: Cache Locality

Modern CPUs are memory-bound. Fetching data from RAM is slow; L1/L2 cache is fast.

Vector: Contiguous memory = Few cache misses.

List: Scattered nodes = Many cache misses.

```
// Iterating 1M elements  
// Fast: Prefetcher friendly  
std::vector<int> v;  
for (int x : v) sum += x;  
// Slow: Pointer chasing  
std::list<int> l;  
for (int x : l) sum += x;
```

The Rule of Thumb

Default to `std::vector`. Only use node-based containers (list, map) if you strictly need pointer stability or $O(1)$ splicing.

Immutability & Thread Safety

Immutability via const

In C++, const objects are immutable. They cannot be modified after initialization. This makes them inherently thread-safe for reading.

```
// Immutable Vector  
const std::vector<int> v = {1, 2, 3};  
// v.push_back(4); // Compile Error!  
  
// Safe to read from multiple threads
```

Thread Safety with Mutexes

For mutable shared state, use `std::mutex` and `std::lock_guard` (RAII) to prevent data races.

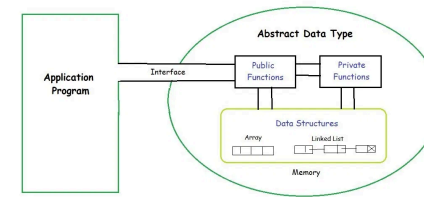
```
std::mutex mu;  
std::vector<int> shared_data;  
  
void add(int x) {  
    std::lock_guard<std::mutex> lock(mu);  
    shared_data.push_back(x);  
}
```

Composition over Inheritance

The "Has-A" Relationship

Build complex ADTs by composing simpler ones. Prefer composition over inheritance to avoid tight coupling and the "fragile base class" problem.

```
class Stack {  
private:  
    // Composition: Stack HAS-A Vector  
    std::vector<int> v;  
  
public:  
    // Delegation  
    void push(int x) {  
        v.push_back(x);  
    }  
};
```



Visualizing Composition: The outer ADT (Stack) wraps the inner ADT (Vector), exposing only the desired interface.

Key Takeaways: The C++ Perspective

01

Abstraction & Encapsulation

Separate *what* (Interface) from *how* (Implementation). Protect internal state to enforce invariants.

02

Safety First

Use **RAII** for resource management and **const correctness** for immutability and thread safety.

03

Performance Awareness

Understand the memory layout. Prefer contiguous storage (`std::vector`) for cache efficiency.

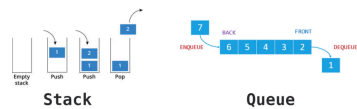
04

Generic Power

Leverage **Templates** to write reusable, type-safe, and zero-overhead ADTs.

Advanced Topics for Further Study

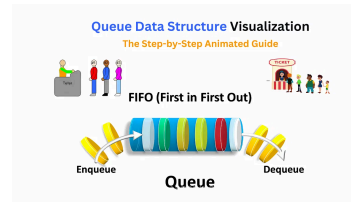
Data Structure Basics



Move Semantics

Optimize ADTs by transferring ownership of resources (pointers) instead of deep copying them. Essential for performance.

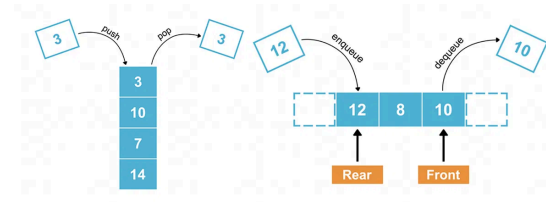
```
MyADT(MyADT&& other);
```



Smart Pointers

Automate memory management within your ADTs using `unique_ptr` and `shared_ptr` to prevent leaks.

```
std::unique_ptr<Node> next;
```



Lock-Free ADTs

Design high-performance concurrent data structures using atomic operations instead of mutexes.

```
std::atomic<Node*> head;
```